

VigiDroid App Workflow: What's Going On?

The `com/msh/vigidroid` package contains the entire logic for the VigiDroid Android application. The workflow describes how the app behaves from user interaction down to machine learning inference.

Step-by-step Workflow

1. App Initialization & User Interface (`MainActivity.java`)

- **Starting Point:** When the user opens the app, `MainActivity` launches.
- **Permissions:** It immediately checks if the app has the "All Files Access" (`MANAGE_EXTERNAL_STORAGE`) permission required for Android 11+. If not, it redirects the user to the Android settings page to grant it. This is necessary because the app needs to read random APK files from the `Downloads` folder.
- **UI Interactions:** The user interface has text views for logs and statuses, and a "Start Scan" button.
- **Triggering a Scan:** When the user clicks the "Start Scan" button, it constructs an `Intent` with an extra `manual_trigger = true`, and starts `ScanService` by calling `ScanService.enqueueWork()`.
- **Listening for Results:** It registers a local `BroadcastReceiver` that listens for the string action `"SCAN_LOG"`. Whenever it receives log broadcasts from the scanning background service, it prints them to the UI text views.

2. Automated Event Triggers (`ApkDownloadReceiver.java`)

- **Background Monitoring:** You don't have to launch the app to scan APKs. The `ApkDownloadReceiver` is a `BroadcastReceiver` configured to listen for system-wide intents (likely `ACTION_DOWNLOAD_COMPLETE` or similar intent filters defined in `AndroidManifest.xml`).
- **Automatic Execution:** When an `.apk` file finishes downloading on the Android phone, this receiver catches the event via its intent data (checking `if (data.toString().endsWith(".apk"))`) and automatically enqueues a job to the background `ScanService` without any direct user interaction.

3. The Scanning Background Service (`ScanService.java`)

- **Initialization (`onCreate`):** When the `ScanService` is spun up, it immediately sets up the machine learning pipeline:
 - It loads the `mh1m_2500_rp_features.json.gz` dictionary of 2500 strings to be used as vector column names.
 - It copies `mh1m_2500_rp_XGBoost.onnx` and the new `1dcnn_model.onnx` from the app's `assets/` folder onto the Android device's physical cache storage.
 - It creates two `OrtSession` (ONNX Runtime) variables into Android's memory, one for XGBoost and one for 1D-CNN.
- **Work Loop (`onHandleWork`):** It verifies the files access permissions again and looks directly into the device's shared `Download` directory for any file ending in `.apk`.
- **Processing Files:** For every APK it finds, it evaluates it using two parallel Machine Learning models:

Pipeline A: XGBoost (Deep Feature Parsing)

- **Parsing the APK details (AxmlReader & MinimalDexParser):** It treats the `.apk` as a standard ZIP file.
 - It looks for `AndroidManifest.xml` and passes it to `AxmlReader.java`. This custom class decodes Android's binary XML format to extract all `android.permission.*` and `android.intent.action.*` strings requested by the APK.
 - It searches for `.dex` (Dalvik Executable code) files and passes them through `MinimalDexParser.java`. This custom class skims Android's compiled bytecode to extract all the native string names of API methods called within the APK.
- **Vectorization:** It checks the massive list of extracted intents, permissions, and method calls against the 2500 known features it loaded previously. It sets `1.0` if the feature is present and `0.0` if missing, resulting in a single `float[2500]` array.
- **Inference:** Passing that `float[]` array through XGBoost's `OrtSession` to predict the malware baseline score.

Pipeline B: 1D-CNN (Shallow Byte Parsing)

- **Byte Extraction:** Using `RandomAccessFile`, it entirely skips structural parsing. Instead, it seeks straight to the bottom of the raw `.apk` file and reads the exact last 1024 bytes.
- **Preprocessing:** Unsigned values are forced to fit within `0-255` integers, and padded with `0` if the whole file is smaller than 1024 bytes. This creates a `long[1024]` array.
- **Inference:** It pushes this `long[]` matrix directly into the PyTorch 1D-CNN's `OrtSession`. The predicted logits are piped through a softmax algorithm to convert the output to a raw `0%` to `100%` malware likelihood score.

4. Hardware Profiling & Reporting

- During these evaluation phases, Android's `SystemClock` and `Debug` modules capture hardware resource consumption:
 - Exact Time in Milliseconds.
 - Thread CPU time (`Debug.threadCpuTimeNanos()`).
 - Native Memory/RAM chunk allocations (`Debug.getNativeHeapAllocatedSize()`).
- All these metrics combined with the malware estimations from the algorithms are formatted into a single log string.
- Finally, it utilizes Android's `LocalBroadcastManager` to push those formatted results straight back up to `MainActivity.java` so you can read them live.
- When finished with all APKs, `onDestroy` tears down memory structures caching the AI to prevent resource leaks.